

Infografía

Módulo 10 — Shaders • Sesión 1

Pintar con matemática: fragment, UV, patrones y uniforms

UPB • Gestión I/2026

Plan de la clase

1. **El GPU no tiene for** — qué es un shader (escena 01).
2. **Dónde vive un shader** — el caminito del inspector.
3. **UV** — cada pixel sabe dónde está (escena 02).
4. **mix** y **smoothstep** — degradados y círculos suaves.
5. **Grafo vs código** — el mismo shader, dos vistas (escena 03).
6. **Patrones** — **floor** + **mod** (escena 04).
7. **Uniforms** — perillas para el inspector (escena 05, en vivo).
8. **Ejercicio** — la bandera (escena 06).

Proyecto: 10._shaders/. Cada escena corre sola con F6.

La pregunta del día

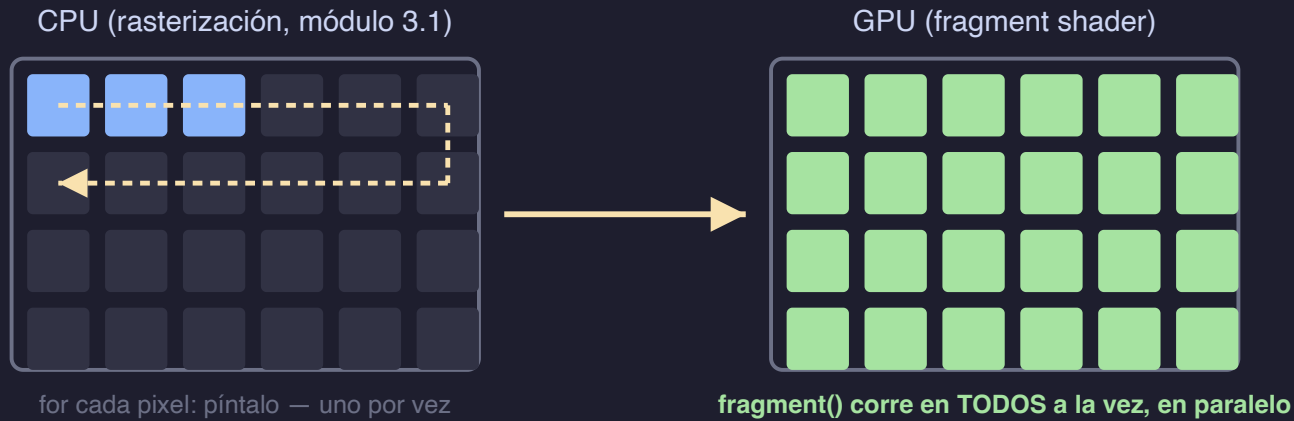
En rasterización (módulo 3.1) pintaste una línea pixel por pixel, con un `for`.

Un juego pinta ~un millón de pixeles, 60 veces por segundo.

¿Cómo? No es un for más rápido. Es NO tener for.

El GPU no tiene for

CPU: un for · GPU: todos a la vez



no hay for: el GPU ya está "dentro" del for
si todos corren el MISMO código, lo único que distingue a un pixel es su posición: UV

Un fragment shader contesta UNA pregunta: ¿de qué color es ESTE pixel?

Tu primer shader (escena 01)

```
shader_type canvas_item;\n\nvoid fragment() {\n    // COLOR es la salida: vec4 (rojo, verde, azul, alfa), 0.0 a 1.0\n    COLOR = vec4(1.0, 0.7, 0.0, 1.0);\n}
```

Tres líneas. `fragment()` corre **para cada pixel**; todos contestan lo mismo → rectángulo sólido.

Perilla: cambia el 0.7 y guarda — el rectángulo cambia al instante. No hay compilar-esperar: el shader recompila al guardar.

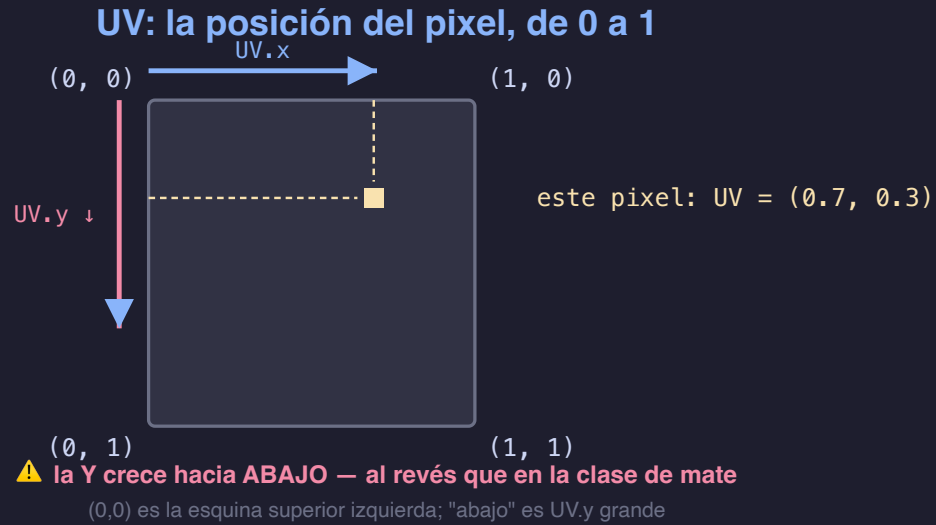
Dónde vive un shader (el caminito)

En un nodo que dibuja (un `ColorRect`, un `Sprite2D`):

1. Inspector → **Material** → `<vacío>` → **Nuevo ShaderMaterial**
2. clic en la bolita del material → **Shader** → `<vacío>` → **Nuevo Shader**
3. doble clic al shader → se abre el **editor de código** abajo
4. escribe → **Ctrl+S** → el resultado cambia en vivo

Esta es la primera pared de los shaders en Godot: no es código difícil, es ENCONTRAR dónde va. En las escenas del proyecto ya está armado.

UV: cada pixel sabe dónde está (escena 02)



```
COLOR = vec4(UV.x, UV.y, 0.0, 1.0); // mismo código, pixel distinto, color distinto
```

¿De qué color es este pixel? • 1

```
void fragment() {  
    COLOR = vec4(UV.y, UV.y, UV.y, 1.0);  
}
```

- A. Degradado negro→blanco, de izquierda a derecha
- B. Degradado negro→blanco, de arriba hacia abajo
- C. Degradado blanco→negro, de arriba hacia abajo
- D. Gris sólido

Vota antes de correrlo. Pista: ¿hacia dónde crece la Y?

mix y smoothstep: las dos herramientas

```
// mix(a, b, t): t=0 da a, t=1 da b, en el medio... la mezcla
COLOR = mix(color_a, color_b, UV.x);           // degradado

// length(UV - 0.5): distancia de este pixel al centro
float dist = length(UV - vec2(0.5));
// smoothstep(a, b, d): 0 si d<a, 1 si d>b, rampa suave entre medio
float t = smoothstep(0.1, 0.5, dist);          // halo radial
```

Distancia + rampa = círculos suaves, viñetas, halos. Lo vas a usar TODO el módulo.

¿De qué color es este pixel? • 2

```
void fragment() {  
    float d = length(UV - vec2(0.5));  
    COLOR = vec4(vec3(step(0.3, d)), 1.0);    // step: 0 si d<0.3, 1 si no  
}
```

- A. Círculo blanco sobre fondo negro
- B. Círculo negro sobre fondo blanco
- C. Anillo blanco
- D. Degradado radial suave

step es smoothstep sin la parte suave: un interruptor.

El grafo y el código son lo mismo (escena 03)

```
GRAFO (VisualShader)
  Input(uv)
    → VectorOp(resta 0.5)
    → VectorLen
    → SmoothStep
    → Output(Color)
```

```
// CÓDIGO (.gdshader)
vec2 c = UV - vec2(0.5);
float d = length(c);
float t =
  smoothstep(0.1, 0.5, d);
COLOR = vec4(vec3(t), 1.0);
```

Cada nodo es una línea. El grafo sirve para explorar; seguimos en código porque se lee, se copia y se versiona. Al GPU le llega lo mismo.

Patrones: floor + mod (escena 04)

```
vec2 celda = floor(UV * 8.0);           // ¿en qué celda caigo? (columna, fila)
float alterna = mod(celda.x + celda.y, 2.0); // 0, 1, 0, 1... tablero
COLOR = vec4(mix(oscuro, claro, alterna), 1.0);
```

La grilla de siempre (flow field, tilemap)... pero **calculada por pixel**, sin nodos ni arrays.

Perilla: cambia el 8.0 — más celdas, gratis. El GPU no se inmuta.

¿De qué color es este pixel? • 3

```
void fragment() {  
    vec2 celda = floor(UV * vec2(8.0, 1.0));  
    COLOR = vec4(vec3(mod(celda.x, 2.0)), 1.0);  
}
```

- A. Franjas verticales blanco/negro
- B. Franjas horizontales blanco/negro
- C. Tablero de ajedrez
- D. Gris uniforme

Mira qué pasa con UV.y: ¿en cuántas celdas se parte?

Los tres errores que vas a ver hoy

Mensajes **reales** del editor — cuando aparezcan, ya los conoces:

```
float x = 1;           // SHADER ERROR: Invalid assignment of 'int' to 'float'.
```

```
COLOR = vec3(1.0, 0.0, 0.0);  
        // SHADER ERROR: Invalid arguments to operator '=': 'vec4, vec3'.
```

```
if (UV.y < 1/3) ...    // SHADER ERROR: Invalid arguments to operator '<': 'float, int'.
```

El idioma del shader NO convierte enteros a flotantes solo. Escribe siempre el punto: `1.0`, `1.0 / 3.0`.
COLOR es vec4: van 4 números.

Uniforms: perillas para el inspector (escena 05 • 🛠️ en vivo)

```
uniform vec3 color_circulo : source_color = vec3(1.0, 0.85, 0.3);  
uniform float radio : hint_range(0.0, 0.7) = 0.25;
```

- `uniform` saca la variable **al inspector** del material.
- El *hint* dice qué perilla dibujar: `source_color` → selector de color, `hint_range(min, max)` → deslizador.
- La próxima sesión: **GDScript escribe estas mismas perillas**.

En vivo: la escena 05 funciona con constantes enterradas. Las convertimos en uniforms y movemos el deslizador con la clase mirando.

🎓 Ejercicio: la bandera (escena 06)

Si la ves **magenta**, falta tu código. En `shaders/exercises/bandera.gdshader`:

1. **Franjas**: rojo / amarillo / verde según el tercio de `UV.y` (if / else if / else).
2. **Círculo blanco suave** en el centro: `length` + `smoothstep` + `mix`.

```
if (UV.y < 1.0 / 3.0) { ... } // ⚠ 1.0 / 3.0 - ¡NO 1/3!
```

Tres niveles de pista en el archivo. Reto extra: que el círculo respire con TIME. Estado roto visible: magenta = no hice nada.

Resumen — la chuleta de hoy

Variable	Qué es	Función	Qué hace
<code>COLOR</code>	salida, vec4 RGBA 0-1	<code>mix(a, b, t)</code>	mezcla $a \rightarrow b$
<code>UV</code>	posición del pixel, 0-1, Y abajo	<code>smoothstep(a, b, x)</code>	rampa suave
<code>TIME</code>	segundos desde el inicio	<code>step(a, x)</code>	interruptor 0/1
<code>uniform</code> + hint	perilla en el inspector	<code>length(v)</code>	distancia
		<code>floor</code> / <code>mod</code>	celdas / alternar

*Un shader contesta: ¿de qué color es ESTE pixel? — usando solo su posición.
La próxima: leer la textura del sprite y manejar el shader DESDE el juego.*